

Bachelor Topic

Dissecting the SWE-bench Pro

Background and Motivation:

Software Engineering (SE) agents, such as SWE-agent [1] and OpenHands [2], are being developed to autonomously resolve repository-level code issues. To evaluate these agents, the research community introduced benchmarks such as SWE-bench [3] and its subsequent variants, including SWE-bench Lite and SWE-bench Verified. However, these benchmarks suffer from data contamination: many evaluated LLMs have likely seen these open-source issues and their pull-request solutions during pre-training, which makes the benchmarks increasingly unreliable for evaluation.

To address this limitation, SWE-bench Pro [4] was introduced to provide a contamination-resistant testbed by shifting the focus to long-horizon SE tasks. Agents are evaluated by generating a patch that is executed against the target repository's test suite and counted as resolved only if it makes previously failing tests pass without breaking existing ones. To this day, the state-of-the-art result achieved on the public dataset remains capped at approximately 59% [5]. The core goal of this research is to analyze the complex issues that current models plus agent fail to resolve, systematically investigate the root causes behind these limitations, and ultimately propose concrete architectural and algorithmic improvements for SE agents.

A similar study was conducted on a smaller scale for SWE-bench Verified by Liu et al. [6], demonstrating how failure analysis can inspire novel multi-agent designs. Because SWE-bench Pro's tasks are larger and span multiple files, they likely surface failure modes that don't appear in Verified's simpler issues.

Student Task and Responsibilities:

- Collect and organize publicly available SWE-bench Pro results generated by state-of-the-art SE agents.
- Compare generated patches against the benchmark's golden patches, analyzing characteristics such as patch size, multi-function changes, multi-file modifications, and implementation complexity.
- Systematically classify failure modes by investigating repository exploration, reasoning process, localization accuracy, and incomplete implementations.
- Perform an empirical analysis to identify common characteristics of unresolved issues and propose improvements for future SE agents based on observed failure patterns.

Deliverables:

- A reproducible analysis pipeline for collecting, comparing, and characterizing SWE-bench Pro submissions against their corresponding golden patches.
- An empirical evaluation of unsuccessful SWE-bench Pro instances, including a taxonomy of failure modes, quantitative analysis of issue and patch characteristics, and recommendations for improving SE agents.

Pre-Requisites: (Programming Languages, OS, Skills, Papers, etc.)

- Experience with Python.

- Familiarity with LLMs and SE agents, including running them within Docker-based test harnesses.
- Basic understanding of Git and patch generation.

[1] Yang, John, et al. "Swe-agent: Agent-computer interfaces enable automated software engineering." Advances in Neural Information Processing Systems 37 (2024): 50528-50652.

[2] Wang, Xingyao, et al. "Openhands: An open platform for AI software developers as generalist agents." International Conference on Learning Representations. Vol. 2025. 2025.

[3] Jimenez, Carlos E., et al. "Swe-bench: Can language models resolve real-world GitHub issues?" International Conference on Learning Representations. Vol. 2024. 2024.

[4] Deng, Xiang, et al. "Swe-bench pro: Can AI agents solve long-horizon software engineering tasks?" arXiv preprint arXiv:2509.16941 (2025).

[5] Scale AI. "SWE-bench Pro Public Leaderboard." Retrieved July 6, 2026, from https://labs.scale.com/leaderboard/swe_bench_pro_public

[6] Liu, Simiao, et al. "An empirical study on failures in automated issue solving." arXiv preprint arXiv:2509.13941 (2025).

Contacts

Thiago Santos de Moura (thiago.santosdemoura@ruhr-uni-bochum.de)
Software Quality Group, Faculty of Computer Science, Ruhr University of Bochum